

All Submissions

Accepted 29 / 29 testcases passed

yousab21 submitted at Mar 08, 2026 19:19

AnalysisSolution

Runtime

8 ms | Beats 82.96%

Memory

11.73 MB | Beats 80.64%

Code | C++

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode(int x) : val(x), next(NULL) {}
7  * };
```

C++ Auto

```
10 //here is use the slow/fast pointers technique
11 //the idea that if there is a loop in the list those pointers will meet
12 //at some point
13 //if there is no loop however the fast pointer will hit a nullptr
14 //in which case we exist the loop and return false
15
16 class Solution {
17 public:
18     bool hasCycle(ListNode *head) {
19         //here the normal edge case is a bit diffrent because in this case
20         //we cant assume a quick return if there is only one node in the list
21         //as this one node can be a loop by pointing ->next to itself
22         if(!head){
23             return false;
24         }
25
26         ListNode* slow = head;
27         ListNode* fast =head->next;
28         while(fast && fast->next){
29             if(fast == slow){
30                 return true;
31             }
32             fast = fast->next->next;
33             slow = slow->next;
34         }
35         return false;
36     }
37 };

```

Saved

Ln 21, Col 70

TestcaseTest Result

Description
Accepted
Editorial
Solutions
Submissions

All Submissions

Accepted
208 / 208 testcases passed

yousab21 submitted at Mar 08, 2026 20:04

Analysis
Solution

Runtime
0 ms | Beats 100.00%

Memory
15.10 MB | Beats 25.09%

Code | C++

```

1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * };
10 */
11
12 struct ListNode {
13     int val;
14     ListNode *next;
15     ListNode() : val(0), next(nullptr) {}
16     ListNode(int x) : val(x), next(nullptr) {}
17     ListNode(int x, ListNode *next) : val(x), next(next) {}
18 };
19
20 class Solution {
21 public:
22     int removeNthFromEnd(ListNode* head, int n) {
23         //first i get the size of the list using this function
24         int size = 1;
25         ListNode* current = head;
26         while(current && current->next){
27             size++;
28             current = current->next;
29         }
30         return size;
31     }
32
33     public:
34     ListNode* removeNthFromEnd(ListNode* head, int n) {
35         //if there is no nodes we just return the empty list
36         //and if there is one node it will be deleted so we also return an empty list
37         if(!head || !head->next){
38             return nullptr;
39         }
40         //here we get the position of the node we want to delete from the front
41         //i add one here because the problem starts counting from 1 not 0
42         int fromFront = getSize(head) - n + 1;
43
44         //i create a dummy node so we have a reference in case we need to delete the head node
45         //and also starting prev from before the head and current from head
46         // makes the for loop a lot more intuitive
47         ListNode* dummy = new ListNode(0, head);
48         ListNode* prev = dummy;
49         ListNode* current = head;
50         for(int i = 1 ; i < fromFront; i++){
51             prev = current;
52             current = current->next;
53         }
54
55         //after the loop current should point to the node we want to delete and prev
56         //to the node before it so we just delete it like normal
57         prev->next = current->next;
58         delete current;
59         return dummy->next;
60     }
61 };

```

View more

More challenges

1721. Swapping Nodes in a Linked List

1474. Delete N Nodes After M Nodes of a Linked List

2095. Delete the Middle Node of a Linked List

Code

C++
Auto

```

11 class Solution {
12     //first i get the size of the list using this function
13     int getSize(ListNode* head){
14         int size = 1;
15         ListNode* current = head;
16         while(current && current->next){
17             size++;
18             current = current->next;
19         }
20         return size;
21     }
22 public:
23     ListNode* removeNthFromEnd(ListNode* head, int n) {
24         //if there is no nodes we just return the empty list
25         //and if there is one node it will be deleted so we also return an empty list
26         if(!head || !head->next){
27             return nullptr;
28         }
29         //here we get the position of the node we want to delete from the front
30         //i add one here because the problem starts counting from 1 not 0
31         int fromFront = getSize(head) - n + 1;
32
33         //i create a dummy node so we have a reference in case we need to delete the head node
34         //and also starting prev from before the head and current from head
35         // makes the for loop a lot more intuitive
36         ListNode* dummy = new ListNode(0, head);
37         ListNode* prev = dummy;
38         ListNode* current = head;
39         for(int i = 1 ; i < fromFront; i++){
40             prev = current;
41             current = current->next;
42         }
43
44         //after the loop current should point to the node we want to delete and prev
45         //to the node before it so we just delete it like normal
46         prev->next = current->next;
47         delete current;
48         return dummy->next;
49     }
50 };

```

Saved

Ln 48, Col 25

Testcase

Test Result

Accepted 19 / 19 testcases passed

yousab21 submitted at Mar 08, 2026 21:09

Analysis

Solution

Runtime

0 ms Beats 100.00%

Memory

15.06 MB Beats 69.89%



Code C++

```
1 /*
2 // Definition for a Node.
3 class Node {
4 public:
5     int val;
6     Node* next;
7     Node* random;
8
```

View more

More challenges

133. Clone Graph

1485. Clone Binary Tree With Random Pointer

1490. Clone N-ary Tree

Code

C++ Auto

```
17
18 class Solution {
19     //i made this function to return a pointer to a node in the second list
20     //that cooresponds to the original node in the old list
21     //it works by walking 2 pointer from the start of each list
22     //until we find the target node in the old list at which point the pointer in
23     //the new list should be pointing to the the node that matches the one in the old list
24     Node* matchInNew(Node* head, Node* newHead, Node* target){
25         if(target == nullptr){
26             return nullptr;
27         }
28         Node *old = head;
29         Node *copy = newHead;
30         while(old != target){
31             old= old->next;
32             copy = copy->next;
33         }
34         return copy;
35     }
36 public:
37     Node* copyRandomList(Node* head) {
38         //usual edge case
39         if(!head){
40             return head;
41         }
42         //dummy node to make keeping track of the head easier
43         Node* dummy = new Node(0);
44
45         //first i create the copy of the list and set all random pointers to null
46         //the reason i do this and cant assine them in the first pass is because
47         //those pointers can point to node next to the one we are at in the old list
48         //that means is shold point to a node that hasnt been created yet in the new list
49         Node* old = head;
50         Node* prev = dummy;
51         Node* current = nullptr;
52         while(old){
53             current = new Node(old->val);
54             prev->next = current;
55             prev = current;
56             old = old->next;
```

Saved

Ln 62, Col 67

Testcase Test Result

Accepted 19 / 19 testcases passed

yousab21 submitted at Mar 08, 2026 21:09

Analysis

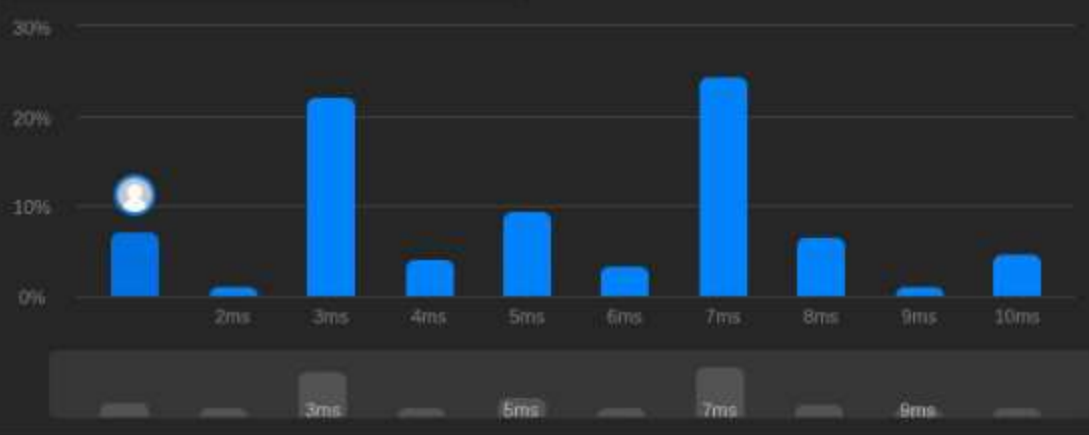
Solution

Runtime

0 ms | Beats 100.00%

Memory

15.06 MB | Beats 69.89%



Code | C++

```
1 /*
2 // Definition for a Node.
3 class Node {
4 public:
5     int val;
6     Node* next;
7     Node* random;
8 }
```

View more

More challenges

133. Clone Graph 1485. Clone Binary Tree With Random Pointer

1490. Clone N-ary Tree

Submit Ctrl Enter

C++ Auto

```
37 Node* copyRandomList(Node* head) {
38     //usual edge case
39     if(!head){
40         return head;
41     }
42     //dummy node to make keeping track of the head easier
43     Node* dummy = new Node(0);
44
45     //first i create the copy of the list and set all random pointers to null
46     //the reason i do this and cant assine them in the first pass is because
47     //those pointers can point to node next to the one we are at in the old list
48     //that means is shold point to a node that hasnt been created yet in the new list
49     Node* old = head;
50     Node* prev = dummy;
51     Node* current = nullptr;
52     while(old){
53         current = new Node(old->val);
54         prev->next = current;
55         prev = current;
56         old = old->next;
57         current = current->next;
58     }
59
60     //now we need to set those random pointer to there correct destination
61     //we start from the heads of both lists and match the pointrs in the new list
62     //to there correpoding in the old list using the function we created
63     old = head;
64     current = dummy->next;
65     while(current){
66         current->random = matchInNew(head,dummy->next,old->random);
67         current = current->next;
68         old = old->next;
69     }
70     return dummy->next;
71 }
72 };
```

Saved

Ln 62, Col 67

Testcase Test Result



Accepted 59 / 59 testcases passed

yousab21 submitted at Mar 09, 2026 19:41

Analysis

Solution

Runtime

0 ms | Beats 100.00%

Memory

65.23 MB | Beats 25.93%



Code | C++

```
1 class Solution {
2 public:
3 //honestly i had to look up the solution for this one i could not
4
5 //here is how this solution works:
6 //since the problem stated that the range of the elements in the array is between 1 and n
7 //and the size of the array is n+1
8 //so we should have n+1 elements each one having a unique element
```

View more

More challenges

Code

C++ Auto

```
1 class Solution {
2 public:
3 //honestly i had to look up the solution for this one i could not figure this on my own
4
5 //here is how this solution works:
6 //since the problem stated that the range of the elements in the array is between 1 and n
7 //and the size of the array is n+1
8 //so we should have n+1 elements each one having a unique element
9 //except if there was no duplicates we will be one element short so there must be 2 element with the same value
10 //now the interesting part :
11 //if we think of the array elements as a linked list nodes and the value of those elements
12 //as the next pointers in the list nodes with the duplicate we would basically have 2 nodes pointing
13 //to one node which means we have a cycle in the list
14 //so we can use same technique with fast and slow pointers (i now know this is an algorithm called Floyd's cycle detection)
15 //to find the beginning of this cycle which should be one of the first of the two duplicate elements
16 int findDuplicate(vector<int>& nums) {
17     int slow = 0 , fast = 0;
18
19     //here is the first loop we walk both pointer until they meet
20     //when they meet we would have detected a cycle
21     //after first loop both pointer would be pointing to some element inside the cycle
22     //but not necessarily the duplicate element however by the way this loop works:
23     //consider :
24     //d = distance from start to cycle entrance
25     //c = cycle length
26     //m = distance from cycle entrance to meeting point
27     //t = distance from the meeting point to the beginning of the cycle
28     //after the first loop pointer move by :
29     //slow : d + m
30     //fast : 2(d + m)
31     //so fast distance - slow distance = 2(d + m) - (d + m) = d + m
32     //also Since they meet at the same node, this difference must be whole number of cycles
33     //so fast distance - slow distance = multipleOf(c)
34     //solving the equations we get : [d = multipleOf(c) - m] -> (1)
35     //and since this is a cycle if you start at the meeting point and move the remaining elements in the cycle
```

Saved

Ln 1, Col 17

Testcase | Test Result



Accepted 59 / 59 testcases passed

Analysis

Solution

yousab21 submitted at Mar 09, 2026 19:41

Runtime

0 ms | Beats 100.00%

Memory

65.23 MB | Beats 25.93%



Code | C++

```
1 class Solution {
2 public:
3 //honestly i had to look up the solution for this one i could not
4
5 //here is how this solution works:
6 //since the problem stated that the range of the elements in the array is from 1 to n+1
7 //and the size of the array is n+1
8 //so we should have n+1 elements each one having a unique element
```

View more

More challenges

Code

C++ v Auto

```
30 // fast : 2(d + m)
31 //so fast_distance - slow_distance = 2(d + m) - (d + m) = d + m
32 //also Since they meet at the same node, this difference must be whole number of cycles
33 //so fast_distance - slow_distance = multipleOf(c)
34 //solving the equations we get : [d = multipleOf(c) - m] ->(1)
35 //and since this is a cycle if you start at the meeting point and move the remaining elements in the cycle
36 //you will reach the beginning of it again so:
37 //t = [multipleOf(c) - m] ->(2)
38 //from one and two we get:
39 // t = d or :
40 //we would reach a state where the distance from the start of the array to the beginning of the cycle
41
42 slow = nums[slow];
43 fast = nums[nums[fast]];
44 while(slow != fast){
45     slow = nums[slow];
46     fast = nums[nums[fast]];
47 }
48
49 //so if we walk the same distance from the meeting point and from the start
50 //when the pointers meet should be the beginning of that cycle aka the duplicate element
51 slow = 0;
52 while (slow != fast) {
53     slow = nums[slow];
54     fast = nums[fast];
55 }
56 return slow;
57 }
58 };
59 //sorry for the large amount of comments iam documenting this for me to return and read
60 //when i eventually get Re-Confused on how this works
```

Saved

Ln 1, Col 17

Testcase Test Result

 **Solution**



View more

- **1670. Design Front Middle Back Queue**

0/5



Ln 95, Col 46

Testcase | **Test Result**

Description Accepted Editorial Solutions Submissions

All Submissions

Accepted 59 / 59 testcases passed

yousab21 submitted at Mar 10, 2026 16:20

Analysis

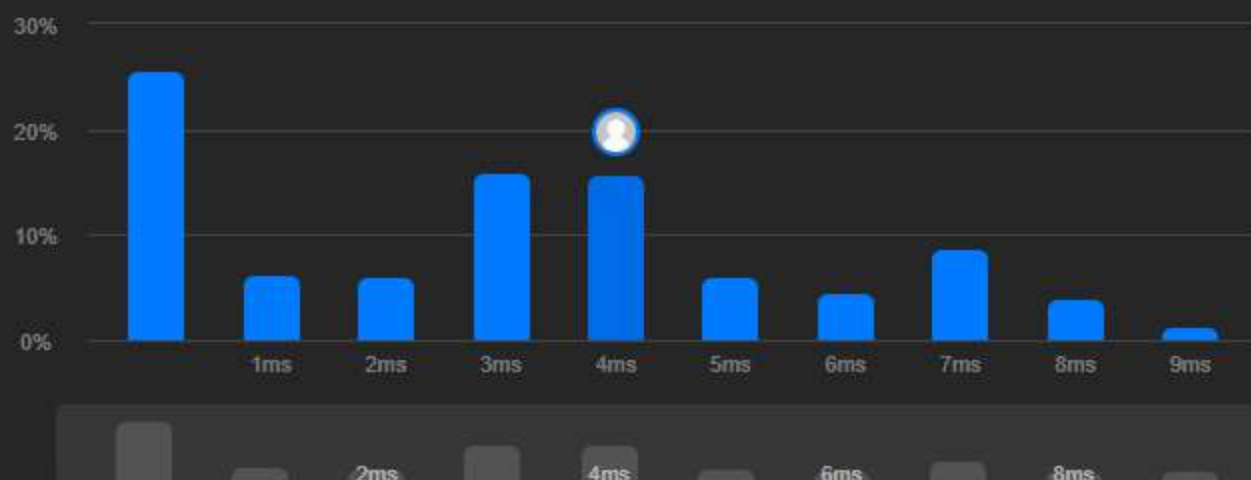
Solution

Runtime

4 ms Beats 46.11%

Memory

23.45 MB Beats 79.96%



Code C++

```
1 class MyCircularQueue {
2 private:
3
4     struct queueNode{
5         int val;
6         queueNode* next;
7         queueNode(int x ,queueNode* y): val(x) , next(y){}
8     };
9 }
```

View more

More challenges

641. Design Circular Deque

1670. Design Front Middle Back Queue

Write your notes here

Select related tags

0/5

Code

C++ Auto

```
46 }
45 //if it is empty i need to make the inial node without moving the rear pointer
44 else if(isEmpty()){
43     rear->val = value;
42     front = rear;
41     size++;
40     return true;
39 }
38 else{ // the usual enqueue operation
37     rear = rear->next;
36     rear->val = value;
35     size++;
34     return true;
33 }
32 }
31 }
30 }
29 bool deQueue() {
28     if(isEmpty()){ //same edge case if the queue is empty the op is invalid
27         return false;
26     }
25     else{// usual dequeue operation
24         front = front->next;
23         size--;
22         return true;
21     }
20 }
19 }
18 int Front() { //if the queue is noe empty i just return the value at front
17     if(isEmpty()){
16         return -1;
15     }
14     return front->val;
13 }
12 }
11 int Rear() { //same with rear
10     if(isEmpty()){
9         return -1;
8     }
7     return rear->val;
6 }
5 }
4 bool isEmpty() { // i just check if size = 0
3     return size == 0;
2 }
1 }
95 bool isFull() { // same if size = maxsize
1     return size == maxSize;
2 }
3 };
```

Saved

Ln 95, Col 46

Testcase

Test Result